Sistema di Gestione per un Negozio di Alimentari

Autore: Gino Pandozzi-Trani

Luglio 2025

Contents

1 Introduzione **3**

2 Analisi dei Requisiti **4**

3 Progettazione Concettuale **5**
3.1 Diagramma ER 5
3.2 Descrizione delle entità 5

4 Progettazione Logica **6**

5 Progettazione Fisica **8**
5.1 Script SQL 8
5.2 Trigger, Funzioni e View 12

6 Popolamento e Dati di Esempio **16**

1 Introduzione

SISTEMA DI GESTIONE PER UN NEGOZIO DI ALIMENTARI Si sviluppi un sistema informativo, composto da una base di dati relazionale e un applicativo Java dotato di GUI (Swing o JavaFX), per la gestione di un sistema di un negozio di ortofrutta. Il sistema deve permettere di gestire le vendite e la disponibilità dei prodotti disponibili, identificando le tipologie (frutta, verdura, farinacei, latticini, uova e confezionati). Ogni tipologia di prodotto ha delle caratteristiche diverse (ad esempio, la frutta e la verdura hanno una data di raccolta, i latticini hanno sia una data di mungitura del latte che una data di produzione, ecc..). Ogni cliente è registrato e ha una tessera punti associata che lo identifica al fruttivendolo. Per ogni acquisto il cliente riceve il 10% del valore della spesa in punti fedeltà. Il sistema deve premettere la ricerca dei clienti, differenziandoli sulla base delle categorie di prodotti acquistati e sulla quantità di punti che hanno ottenuto per ogni categoria. Per il gruppo da 3: il fruttivendolo ha un certo numero di dipendenti, e ad ogni vendita è associato il dipendente che l'ha portata a termine. Deve pertanto essere possibile ricercare quale dipendente ha effettuato il maggior numero di vendite in un determinato periodo differenziandolo dal dipendente che ha portato al fruttivendolo il maggior introito. Breve presentazione del progetto, obiettivi e contesto della traccia accademica.

2 Analisi dei Requisiti

- Gestione prodotti per tipologia (frutta, verdura, farinacei, latticini, uova, confezionati)
- Gestione clienti e tessere punti
- Gestione dipendenti e vendite
- Statistiche e ricerche

3 Progettazione Concettuale

3.1 Diagramma ER

3.2 Descrizione delle entità

4 Progettazione Logica

Schema relazionale
Lo schema relazionale rappresenta la struttura del database, evidenziando le relazioni tra le tabelle e i vincoli di integrità. Nella tabella seguente sono indicate le chiavi primarie (sottolineate) e le chiavi esterne (doppia sottolineatura), secondo la convenzione accademica.

Tabella	Attributi
cliente	codcliente (PK), nome, cognome, codicefiscale, indirizzo, telefono, email
tessera	codtessera (PK), codcliente (FK, UNIQUE), numeropunti, dataemissione, datascadenza, stato
dipendente	coddipendente (PK), nome, cognome, codicefiscale, indirizzo, telefono, email
prodotto	codprodotto (PK), nome, descrizione, prezzo, luogoprovenienza, dataraccolta, datamungitura, glutine, datascadenza, dataproduzione, categoria, scorta
ordine	codordine (PK), prezzototale, dataacquisto, codcliente (FK), coddipendente (FK)
articoliordine	codordine (PK, FK), codprodotto (PK, FK), prezzo, numeroarticoli

Table 1: Schema relazionale: chiavi primarie sottolineate, chiavi esterne doppiamente sottolineate.

Dizionario delle classi
Il dizionario delle classi descrive le principali entità del database, specificando il ruolo di ciascuna tabella nel sistema informativo.

Classe (Tabella)	Descrizione
cliente	Dati anagrafici dei clienti registrati nel sistema.
tessera	Tessera punti associata a ciascun cliente, saldo punti e stato.
dipendente	Dati anagrafici dei dipendenti del negozio.
prodotto	Prodotti in vendita, con tipologia, caratteristiche e scorta.
ordine	Vendita, associata a cliente e dipendente.
articoliordine	Dettaglio dei prodotti acquistati in ogni ordine.

Table 2: Classi principali del database e loro funzione.

Associazioni tra le entità

Le associazioni rappresentano i legami logici tra le entità del database, utili per comprendere la struttura delle relazioni.

Associazione	Descrizione
Cliente-Tessera	Ogni cliente ha una tessera (1:1).
Ordine-Cliente	Ogni ordine è associato a un cliente (N:1).
Ordine-Dipendente	Ogni ordine è associato a un dipendente (N:1).
Ordine-Prodotto	Ogni ordine contiene più prodotti tramite articoliordine (N:M).

Table 3: Principali associazioni tra le entità del database.

Vincoli e regole di integrità

Questa sezione riassume i vincoli e le regole di integrità che garantiscono la coerenza e la correttezza dei dati nel database.

Nome Vincolo	Descrizione
Email Regex	Gli indirizzi email di clienti e dipendenti devono essere di forma legittima: almeno un carattere prima della @, almeno un carattere tra questa ed il punto, almeno un carattere dopo il punto.
Codice Fiscale Regex	Il codice fiscale deve rispettare il formato italiano standard.
Chiavi Primarie	Ogni tabella ha una chiave primaria che garantisce l'unicità dei record.
Chiavi Esterne	Le chiavi esterne garantiscono l'integrità referenziale tra le tabelle (es. Tessera-CodCliente, Ordine-CodCliente, CompOrdine-CodOrdine, ecc.).
Vincoli CHECK	Su valori numerici (es. scorta ≥ 0, prezzo ≥ 0), presenza di dati obbligatori, coerenza attributi per categoria prodotto.

Table 4: Vincoli e regole di integrità implementati nel database.

Schema logico: trigger e procedure

Infine, vengono presentati i principali trigger e procedure che automatizzano la gestione e il controllo dei dati, migliorando la robustezza del sistema.

Trigger/Procedura	Descrizione
VerificaScortaProdotto	Blocca l'inserimento di articoli in ordine se la scorta è insufficiente.
AggiornaScortaEPunti	Aggiorna la scorta del prodotto e accredita i punti fedeltà al cliente dopo l'inserimento di un articolo.
AggiornaPrezzoOrdine	Aggiorna il prezzo totale dell'ordine dopo l'inserimento di un articolo.
AggiornaStatoTessera	Aggiorna automaticamente lo stato della tessera in base alla data di scadenza.
RipristinaScortaProdott	Ripristina la scorta e ricalcola i punti dopo la cancellazione di un articolo da un ordine.
AggiornaPrezzoOrdineDelete	Aggiorna il prezzo totale dell'ordine dopo la cancellazione di un articolo.

Table 5: Trigger e procedure principali implementate per la gestione automatica dei dati.

5 Progettazione Fisica

5.1 Script SQL

Enumerazioni

```
-- Creare il nuovo tipo ENUM con le categorie corrette e
   migliorato
CREATE TYPE TIPOLOGIA AS ENUM (
    'FRUTTA',
    'VERDURA',
    'FARINACEI',
    'LATTICINI',
    'UOVA',
    'CONFEZIONATI'
);

-- Creare tipo ENUM per lo stato della tessera
CREATE TYPE STATOTESSERA AS ENUM (
    'ATTIVA',
    'SOSPESA',
    'SCADUTA'
);
```

Tabella cliente

```
CREATE TABLE IF NOT EXISTS cliente (
    codcliente SERIAL PRIMARY KEY,
```

```
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* '^[A-Za-z ]+$
       '),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
       ~* '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* '^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* '^[A-Za-z0
       -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella tessera

```
CREATE TABLE IF NOT EXISTS tessera (
    codtessera SERIAL PRIMARY KEY,
    codcliente INTEGER NOT NULL UNIQUE,
    numeropunti NUMERIC NOT NULL DEFAULT 0.00 CHECK (numeropunti
       >= 0.00),
    dataemissione DATE NOT NULL DEFAULT CURRENT_DATE,
    datascadenza DATE NOT NULL DEFAULT (CURRENT_DATE + INTERVAL '
       2 years'),
    stato STATOTESSERA NOT NULL DEFAULT 'ATTIVA',
    CONSTRAINT tessera_cliente_fk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Tabella dipendente

```
CREATE TABLE IF NOT EXISTS dipendente (
    coddipendente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* '^[A-Za-z ]+$
       '),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
       ~* '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* '^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* '^[A-Za-z0
       -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella prodotto

```
CREATE TABLE IF NOT EXISTS prodotto (
    codprodotto SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z ]+$'),
    descrizione VARCHAR(500),
    prezzo NUMERIC DEFAULT 0.00 CHECK (prezzo >= 0.00),
    luogoprovenienza VARCHAR(255),
    dataraccolta DATE,
    datamungitura DATE,
    glutine BOOLEAN,
    datascadenza DATE,
    dataproduzione DATE,
    categoria TIPOLOGIA,
    scorta INT CHECK (scorta >= 0)
);
```

Vincolo CHECK su prodotto

```
ALTER TABLE prodotto ADD CONSTRAINT checkCategoria CHECK (
  -- FRUTTA: deve avere data di raccolta
  (categoria = 'FRUTTA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- VERDURA: deve avere data di raccolta
  (categoria = 'VERDURA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- LATTICINI: devono avere data di mungitura del latte e data
     di produzione
  (categoria = 'LATTICINI' AND dataraccolta IS NULL AND
     datamungitura IS NOT NULL AND dataproduzione IS NOT NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL) OR

  -- FARINACEI: informazioni sul glutine
  (categoria = 'FARINACEI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NOT NULL) OR

  -- UOVA: solo data di scadenza
  (categoria = 'UOVA' AND dataraccolta IS NULL AND datamungitura
     IS NULL AND dataproduzione IS NULL AND datascadenza IS NOT
     NULL AND glutine IS NULL) OR

  -- CONFEZIONATI: solo data di scadenza
```

```
  (categoria = 'CONFEZIONATI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL)
);
```

Tabella ordine

```
CREATE TABLE IF NOT EXISTS ordine (
    codordine SERIAL PRIMARY KEY,
    prezzototale REAL NOT NULL DEFAULT 0.00 CHECK (prezzototale
       >= 0),
    dataacquisto DATE NOT NULL,
    codcliente INTEGER NOT NULL,
    coddipendente INTEGER NOT NULL,
    CONSTRAINT ordineclientefk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    CONSTRAINT ordinedipendentefk FOREIGN KEY (coddipendente)
       REFERENCES dipendente (coddipendente)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);
```

Tabella articoliordine

```
CREATE TABLE IF NOT EXISTS articoliordine (
    codordine INTEGER NOT NULL,
    codprodotto INTEGER NOT NULL,
    prezzo NUMERIC NOT NULL DEFAULT 0.00 CHECK (prezzo >= 0.00),
    numeroarticoli INT NOT NULL CHECK (numeroarticoli > 0),
    CONSTRAINT PK_ArticoliOrdine PRIMARY KEY (codordine,
       codprodotto),
    CONSTRAINT articoliordineprodottofk FOREIGN KEY (codprodotto)
        REFERENCES prodotto (codprodotto)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    CONSTRAINT articoliordineordinek FOREIGN KEY (codordine)
       REFERENCES ordine (codordine)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

5.2 Trigger, Funzioni e View

Trigger principali

```
-- Trigger per aggiornare scorta e punti tessera dopo inserimento
    articolo
CREATE TRIGGER TrgAggiornaScortaPunti
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaScortaEPunti();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   inserimento articolo
CREATE TRIGGER TrgAggiornaPrezzoOrdine
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdine();

CREATE TRIGGER TrgVerificaScortaProdotto
BEFORE INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION VerificaScortaProdotto();

-- Trigger per aggiornare la scorta e i punti tessera dopo
   inserimento articolo

CREATE TRIGGER TrgAggiornaStatoTessera
BEFORE INSERT OR UPDATE ON tessera
FOR EACH ROW
EXECUTE FUNCTION AggiornaStatoTessera();

-- Trigger per ripristinare scorta e punti tessera dopo DELETE da
    articoliordine
CREATE TRIGGER TrgRipristinaScortaPunti
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION RipristinaScortaProdotto();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   DELETE da articoliordine
CREATE TRIGGER TrgAggiornaPrezzoOrdineDelete
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdineDelete();
```

Funzione AggiornaScortaEPunti

```
CREATE OR REPLACE FUNCTION AggiornaScortaEPunti()
RETURNS TRIGGER AS $$
DECLARE
    punti_da_aggiungere NUMERIC;
    cod_cliente INTEGER;
BEGIN
    -- Aggiorna la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta - NEW.numeroarticoli
    WHERE codprodotto = NEW.codprodotto;

    -- Calcola i punti da aggiungere (10% del valore speso)
    punti_da_aggiungere := NEW.prezzo * NEW.numeroarticoli *
       0.10;

    -- Recupera il cliente associato all'ordine
    SELECT codcliente INTO cod_cliente
    FROM ordine
    WHERE codordine = NEW.codordine;

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = numeropunti + punti_da_aggiungere
    WHERE codcliente = cod_cliente;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdine

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdine()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = prezzototale + (NEW.prezzo * NEW.
       numeroarticoli)
    WHERE codordine = NEW.codordine;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione VerificaScortaProdotto

```
CREATE OR REPLACE FUNCTION VerificaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    scorta_corrente INT;
BEGIN
    SELECT scorta INTO scorta_corrente FROM prodotto WHERE
       codprodotto = NEW.codprodotto;
    IF scorta_corrente < NEW.numeroarticoli THEN
        RAISE EXCEPTION 'Scorta insufficiente per il prodotto %:
           richiesta % - disponibile %', NEW.codprodotto, NEW.
           numeroarticoli, scorta_corrente;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaStatoTessera

```
CREATE OR REPLACE FUNCTION AggiornaStatoTessera()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.datascadenza < CURRENT_DATE THEN
        NEW.stato := 'SCADUTA';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione RipristinaScortaProdotto

```
CREATE OR REPLACE FUNCTION RipristinaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    cod_cliente INTEGER;
    punti_da_togliere NUMERIC;
BEGIN
    -- Ripristina la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta + OLD.numeroarticoli
    WHERE codprodotto = OLD.codprodotto;

    -- Recupera il cliente associato all'ordine
    SELECT codcliente INTO cod_cliente
    FROM ordine
```

```
    WHERE codordine = OLD.codordine;

    -- Calcola i punti da togliere (10% del valore speso)
    punti_da_togliere := OLD.prezzo * OLD.numeroarticoli * 0.10;

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = GREATEST(numeropunti - punti_da_togliere,
       0)
    WHERE codcliente = cod_cliente;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdineDelete

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdineDelete()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = GREATEST(prezzototale - (OLD.prezzo * OLD.
       numeroarticoli), 0)
    WHERE codordine = OLD.codordine;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

6 Popolamento e Dati di Esempio

Il popolamento del database è stato effettuato tramite lo script `Popolazione.sql`, che inserisce dati di esempio per tutte le principali entità del sistema:

- Prodotti di tutte le tipologie (frutta, verdura, farinacei, latticini, uova, confezionati), con relative caratteristiche specifiche.
- Clienti con tessere punti associate e saldo iniziale.
- Dipendenti e dati anagrafici.
- Vendite, ciascuna associata a cliente, dipendente e prodotti acquistati.
- Movimenti punti fedeltà generati dalle vendite.

I dati inseriti permettono di testare tutte le funzionalità implementate, comprese le query di ricerca, le statistiche e la gestione dei punti fedeltà. Lo script può essere eseguito dopo la creazione delle tabelle per ottenere un database pronto all'uso con dati realistici.